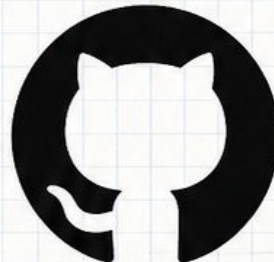


GIT & GITHUB PRODUCTION HANDBOOK

FROM FUNDAMENTALS TO ENTERPRISE
PRODUCTION WORKFLOWS



20
PAGES

COMPLETE CURRICULUM

- ✓ Production-Ready Workflows
- ✓ Enterprise Best Practices
- ✓ CI/CD with GitHub Actions
- ✓ Security & Compliance
- ✓ Real-World Labs
- ✓ Troubleshooting & Recovery

VERSION CONTROL. COLLABORATE. AUTOMATE. DEPLOY.
BUILD BETTER. TOGETHER.



FUNDAMENTALS
TO ADVANCED



TEAM
COLLABORATION



SECURE
& RELIABLE



AUTOMATE
EVERYTHING



DEPLOY
WITH CONFIDENCE

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



1. INTRODUCTION TO GIT & GITHUB



1.1 WHAT GIT IS

- Git is a distributed version control system designed to track changes in source code and collaborate efficiently.
- Created by Linus Torvalds in 2005.
- Fast, reliable, and built for large-scale projects.

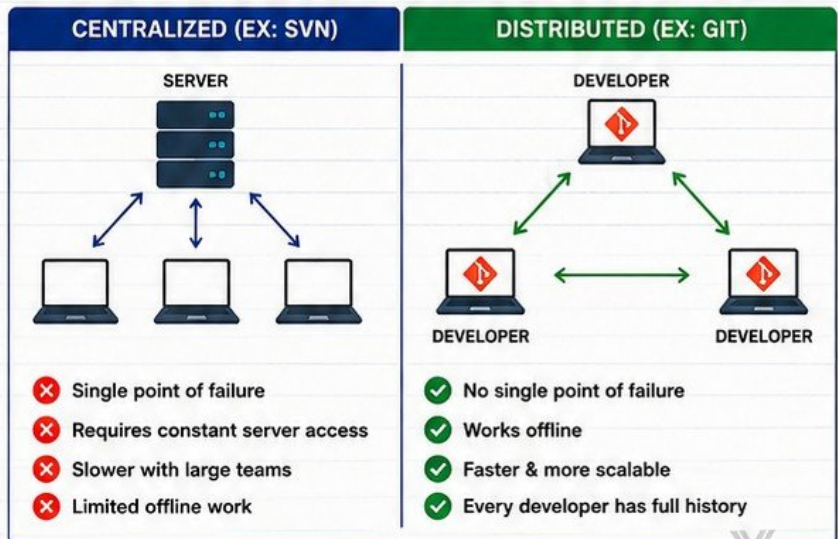
1.2 VERSION CONTROL FUNDAMENTALS

- Tracks every change to files over time.
- Allows multiple developers to work on the same project.
- Maintains a complete history.
- Enables safe experimentation.
- Supports branching and merging.

1.3 WHY GIT POWERS MODERN SOFTWARE ENGINEERING

- Handles large codebases with speed.
- Enables distributed collaboration.
- Supports branching, merging, and parallel development.
- Integrates seamlessly with DevOps and CI/CD pipelines.
- Used by millions of engineers worldwide.

1.4 DISTRIBUTED VS CENTRALIZED VERSION CONTROL



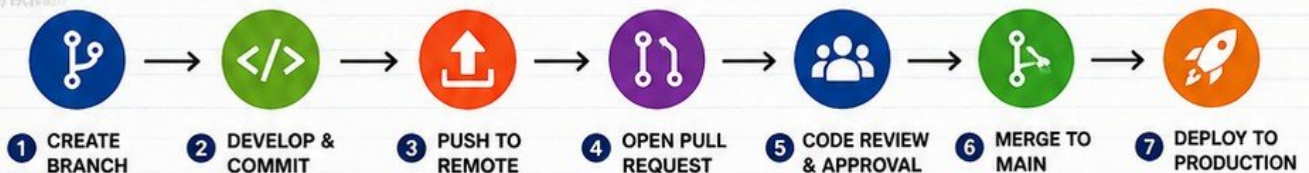
1.5 HANDBOOK OBJECTIVES

- ✓ Master Git fundamentals for production.
- ✓ Use Git & GitHub like a senior engineer.
- ✓ Implement enterprise-ready workflows.
- ✓ Collaborate securely and efficiently.
- ✓ Integrate Git into CI/CD and automation.
- ✓ Troubleshoot and solve real-world issues.

REMEMBER:

GOOD VERSION CONTROL PRACTICES LEAD TO BETTER CODE, BETTER COLLABORATION, AND BETTER PRODUCTS.

1.6 ENTERPRISE DEVELOPMENT WORKFLOW (HIGH LEVEL)



GIT IS NOT JUST A TOOL. IT IS THE FOUNDATION OF MODERN SOFTWARE COLLABORATION.
MASTER IT. USE IT. TEACH IT.

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



2. INSTALLING AND CONFIGURING GIT



2.1 GIT INSTALLATION

PLATFORM	INSTALL COMMAND / ACTION
Windows	Download from https://git-scm.com/download/win
macOS	brew install git or download from https://git-scm.com/download/mac
Linux	sudo apt install git (Debian/Ubuntu) sudo yum install git (RHEL/CentOS)

2.2 GLOBAL CONFIGURATION

Set your identity and preferences once for your system.

```
# Set your username
git config --global user.name "Your Name"

# Set your email
git config --global user.email "you@example.com"

# Check your configuration
git config --global --list
```

2.3 USERNAME AND EMAIL

These values will be attached to every commit you make.



TIP: Use a professional email that you will retain long-term.

2.7 USEFUL GIT SETTINGS

Set default pull behavior	Show colored output	Set automatic line endings	Show branch ahead/behind info	Better diff output	Show relative timestamps
git config --global pull.rebase false (merge by default)	git config --global color.ui auto (improves readability)	git config --global core.autocrlf true (Windows users)	git config --global status.aheadBehind true	git config --global diff.tool vimdiff	git config --global log.date relative

2.4 DEFAULT BRANCH CONFIGURATION

Set the default branch name for new repositories.

```
# Set default branch to main
git config --global init.defaultBranch main
```

2.5 SSH VS HTTPS

SSH	HTTPS
- More secure - Uses SSH keys - Ideal for frequent pushes - Required for GitHub Actions - Example: git@github.com:user/repo.git	- Easy to use - Uses username/password or PAT - Good for beginners - Example: https://github.com/user/repo.git

2.6 GIT CONFIGURATION FILES

LEVEL	FILE LOCATION	SCOPE	DESCRIPTION
System	/etc/gitconfig	All Users	System-wide settings
Global	~/.gitconfig (Windows: ~/.gitconfig)	Current User	User-wide settings
Local	.git/config (in repository)	Repository	Repository-specific settings

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



3. GIT REPOSITORY FUNDAMENTALS



3.1 INITIALIZE REPOSITORIES

Create a new Git repository in an existing project directory.

```
$ git init
```

Initialized empty Git repository in
./my-project/.git/

3.2 CLONE REPOSITORIES

Create a local copy of a remote repository.

```
# Using HTTPS
```

```
$ git clone https://github.com/user/repo.git
```

```
# Using SSH
```

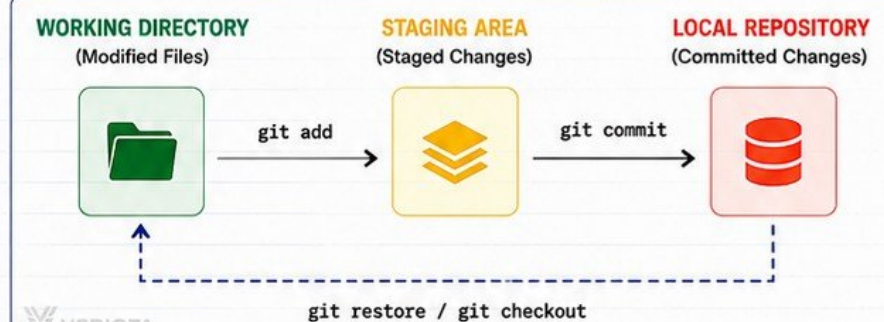
```
$ git clone git@github.com:user/repo.git
```

3.3 REPOSITORY STRUCTURE

.git/	Git metadata and object database
Files	Your project files and directories
.gitignore	Files and patterns ignored by Git
README.md	Project documentation and overview

3.4 THE THREE AREAS OF GIT

- WORKING DIRECTORY**
Contains your project files. You edit and modify files here.
- STAGING AREA**
A preparation area where you stage changes before committing.
- LOCAL REPOSITORY**
Stores your committed changes in the local .git database.



3.5 REPOSITORY LIFECYCLE



1. INIT
Start a new repository

```
git init
```



2. CLONE
Copy an existing repository

```
git clone
```



3. MODIFY
Make changes in your working directory



4. STAGE
Add changes to the staging area

```
git add
```



5. COMMIT
Save changes to local repository

```
git commit
```



6. PUSH
Send changes to remote repository

```
git push
```

3.6 KEY TAKEAWAYS

- Git tracks changes in files over time.
- Changes move from Working Directory → Staging Area → Local Repository.
- Commits in the Local Repository create a permanent history.
- Repositories can be local or remote.
- These fundamentals power all Git workflows.

QUICK REFERENCE COMMANDS

<code>git init</code>	Initialize a new repository
<code>git clone <url></code>	Clone a remote repository
<code>git status</code>	Show changes
<code>git add <file></code>	Stage changes
<code>git commit -m "msg"</code>	Commit changes
<code>git log</code>	Show commit history

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



4. RECORDING CHANGES



Git records changes in snapshots called commits.
These commits build the history of your project.

1 4.1 GIT STATUS

Shows the current state of your working directory and staging area.

```
$ git status
```

On branch main

Changes not staged for commit:

```
modified:   app.js
```

Changes to be committed:

```
new file:   index.html
```



Use git status frequently to understand what's changed.

2 4.2 GIT ADD

Stages changes to be included in the next commit.

```
$ git add <file> # stage a file
$ git add .       # stage all changes in current directory
$ git add -p      # interactively stage hunks
```



Staged changes move from the Working Directory to the Staging Area.

3 4.3 GIT COMMIT

Records staged changes in the local repository.

```
$ git commit -m "Add user login feature"
$ git commit -am "Fix session expiration bug"
```

- -m : add commit message
- -a : stage tracked files automatically
- -am : stage + commit tracked files



A commit is a permanent snapshot of your project at a point in time.

4 4.4 COMMIT MESSAGES

A good commit message explains WHY the change was made.

✓ GOOD EXAMPLE

Add password reset endpoint
Implement POST /api/auth/reset endpoint with token validation, email sending, and rate limiting.

Why:
Allows users to securely reset their password if they forget it.

Closes #123

✗ POOR EXAMPLE

fix bug

update

changes

test

WIP

5 4.5 COMMIT BEST PRACTICES

- ✓ Use the imperative mood (Add, Fix, Update, Remove)
- ✓ Keep the first line under 50 characters
- ✓ Provide more detail in the body if needed
- ✓ Explain "what" and "why", not just "how"
- ✓ Reference issues or tickets (e.g., Closes #123)
- ✓ Keep commits focused and meaningful
- ✓ Avoid committing generated files or secrets

6 4.6 ATOMIC COMMITS

Each commit should represent ONE logical change.

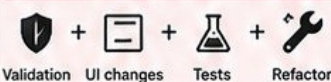
✓ ATOMIC (GOOD)



Small, focused, easy to review and revert.

✗ NOT ATOMIC (BAD)

Big commit with many unrelated changes



Hard to review, debug and revert.

7 4.7 COMMIT HISTORY

View the history of commits in your repository.

```
$ git log # full history
$ git log --oneline # compact view
$ git log --graph --all --decorate # with graph
$ git show <commit-hash> # details
```

```
$ git log --oneline --graph --all
* a1b2c3d (HEAD -> main) Add login tests
* b2c3d4e Update login UI
* c3d4e5f Add login validation
| \
| * d4e5f6g (feature/reset) Add reset endpoint
*/ e5f6g7h Initial commit
```



REMEMBER

Stage carefully. Commit intentionally. Write meaningful messages.

GOOD COMMITS = CLEAN HISTORY = BETTER COLLABORATION

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



5. VIEWING HISTORY AND COMPARING CHANGES



Understand what changed, when, why, and by whom.
Git gives you powerful tools to explore your project history.

1 5.1 GIT LOG

View the commit history.

```
$ git log                # full history
$ git log --oneline      # compact view
$ git log --graph --oneline --all
$ git log --author="John Doe"
$ git log --since="2024-01-01"
$ git log --stat         # with stats
```



Use git log to understand the timeline and evolution of your project.

2 5.2 GIT SHOW

Show details of a specific commit.

```
$ git show <commit-hash>          # show details
$ git show --stat <commit-hash>   # with stats
$ git show --name-only <commit-hash> # changed files
```



Shows the metadata, changes, and author of a commit.

3 5.3 GIT DIFF

Compare changes between states.

```
$ git diff                # unstaged changes
$ git diff --staged       # staged vs last commit
$ git diff <commit1> <commit2>
$ git diff main..feature  # compare commits
$ git diff --word-diff    # word-level diff
```



See exact line-by-line changes between versions.

4 5.4 COMMIT COMPARISON

Compare two commits to see what changed between them.

```
$ git log --oneline
* e5f6c3a (HEAD -> main) Fix login bug
* b2c3d4e Add user validation
* a1b2c3d Implement login feature
* 9f8e7d6 Initial commit
```

COMPARE EXAMPLE

Compare a1b2c3d and e5f6c3a

```
$ git diff a1b2c3d e5f6c3a
```

Compare main and feature branches

```
$ git diff main..feature
```

DIFF OUTPUT (EXAMPLE)

```
@@ -1,3 +1,3 @@
function login(user, pass) {
-  if(user && pass) {
+  if(user && pass && isValid(user)) {
    return true;
  }
```

5 5.5 FILE COMPARISON

Compare specific files between commits, branches, or working states.



Compare file between working directory and staged

```
$ git diff -- <file>
```



Compare file between staged and last commit

```
$ git diff --staged -- <file>
```



Compare file between two commits

```
$ git diff <commit1> <commit2> -- <file>
```



Compare file between two branches

```
$ git diff <branch1>..<branch2> -- <file>
```



TIP

Use a dot (.) for the current directory or a specific file path.

Example:

```
$ git diff main..feature -- src/app.js
```

6 5.6 BLAME ANALYSIS

Find out who changed each line and why.

```
$ git blame <file>                # current file
$ git blame -L 10,20 <file>       # specific lines
$ git blame -C -C <file>          # detect moved code
```

```
e5f6c3a1 (John) 10  const express = require('express');
b2c3d4e2 (Jane) 11  const app = express();
e5f6c3a1 (John) 12  app.use(authMiddleware);
a1b2c3d3 (Alex) 13  app.listen(3000);
```



Git blame helps during debugging, code reviews, and understanding code ownership.

7 5.7 HISTORY INVESTIGATION

Investigate history to answer important questions.



Who introduced a bug?

```
→ git log -S "buggyFunction"
```



When was a feature added?

```
→ git log --since="2024-06-01"
```



Which commit changed this line?

```
→ git blame <file>
```



When was a file deleted or moved?

```
→ git log --follow <file>
```



Which commit added this function?

```
→ git log -G "functionName"
```



REMEMBER

The history is your source of truth.
Explore it before you change it.

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



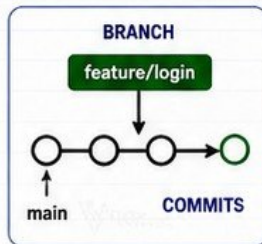
6. BRANCHING STRATEGIES



Branches let you work in parallel, isolate changes, and deliver software safely and efficiently.

1 6.1 BRANCH CONCEPTS

- Branches point to commits.
- Each branch is a movable pointer.
- Branches allow isolated work.
- Changes merge back into main.
- Branches enable collaboration.

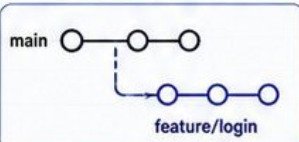


2 6.2 TYPES OF BRANCHES

Feature Branch	Build new features or changes.
Release Branch	Prepare for production releases.
Hotfix Branch	Fix critical issues in production.
Long-lived Branch	Exist for a long time (e.g., main, develop).
Short-lived Branch	Created for a short time and merged.

3 6.3 FEATURE BRANCHES

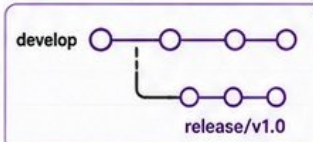
Used for developing new features or large changes.



```
# create
git checkout -b feature/login
# work and commit
# merge
git checkout main
git merge feature/login
# delete
git branch -d feature/login
```

4 6.4 RELEASE BRANCHES

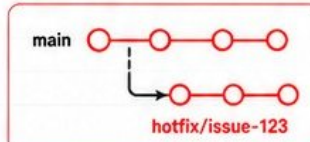
Stabilize code before production release.



```
# create
git checkout -b release/v1.0
# bug fixes, docs, tests
# merge to main and develop
git checkout main
git merge release/v1.0
git checkout develop
git merge release/v1.0
# delete
git branch -d release/v1.0
```

5 6.5 HOTFIX BRANCHES

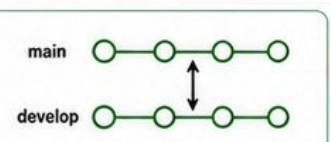
Quick fixes for critical issues in production.



```
# create from main
git checkout -b hotfix/issue-123
# fix issue and commit
# merge to main
git checkout main
git merge hotfix/issue-123
# (optional) merge to develop
git checkout develop
git merge hotfix/issue-123
# delete
git branch -d hotfix/issue-123
```

6 6.6 LONG-LIVED BRANCHES

Branches that exist for the entire lifecycle of the project.

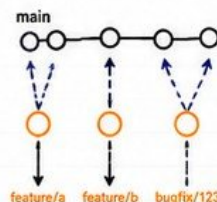


- ✓ Examples: main, develop, staging
- ✓ Rarely deleted
- ✓ Base for other branches
- ✓ Protected with strict rules
- ✓ Require PR reviews
- ✓ Represent stable lines of development

7 6.7 SHORT-LIVED BRANCHES

Created for short tasks and quickly merged.

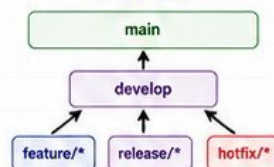
- ✓ Examples: feature/*, bugfix/*, chore/*
- ✓ Live for hours or days
- ✓ Reduce merge conflicts
- ✓ Keep history clean
- ✓ Improve focus and collaboration



8 6.8 ENTERPRISE BRANCHING MODELS

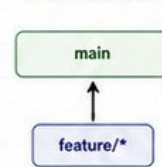
Common models used in real-world teams.

A. GIT FLOW



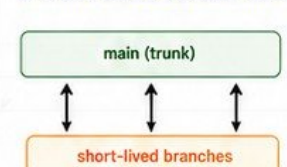
- ✓ Strong structure and rules
- ✓ Best for large teams
- ✓ More branches, more overhead
- ✓ Good for compliance and releases

B. GITHUB FLOW



- ✓ Simple and fast
- ✓ Feature branches
- ✓ Pull request → merge
- ✓ Deploy anytime

C. TRUNK-BASED DEVELOPMENT



- ✓ Very short-lived branches
- ✓ Continuous integration
- ✓ Small changes frequently
- ✓ Ideal for high-velocity teams



KEY TAKEAWAYS

- ✓ Use branches to isolate work and reduce risk.
- ✓ Keep branches short-lived when possible.
- ✓ Protect long-lived branches.
- ✓ Choose a branching model that fits your team and workflow.



TIP

A good branching strategy prevents chaos, reduces conflicts, and enables safe, fast delivery of high-quality software.

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

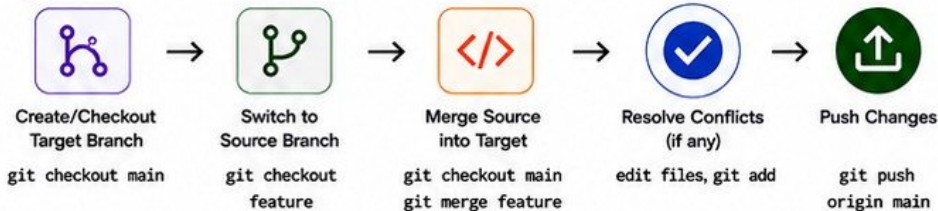


7. MERGING AND REBASING



Combine changes from different branches
and keep your history clean and meaningful.

1 7.1 MERGE WORKFLOW



MERGE COMMAND

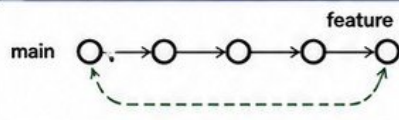
```
# Merge feature into main
git checkout main
git merge feature/login
```



Merging preserves the full history of all branches and commits.

2 7.2 FAST-FORWARD MERGE

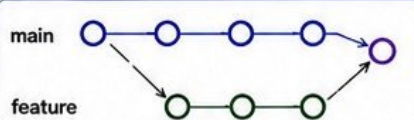
Occurs when the target branch has not diverged from the source branch.



```
# Fast-forward merge
git checkout main
git merge feature
# no merge commit created
```

3 7.3 THREE-WAY MERGE

Occurs when both branches have diverged.



```
# Three-way merge
git checkout main
git merge feature
# creates a merge commit
```

THREE-WAY MERGE EXAMPLE

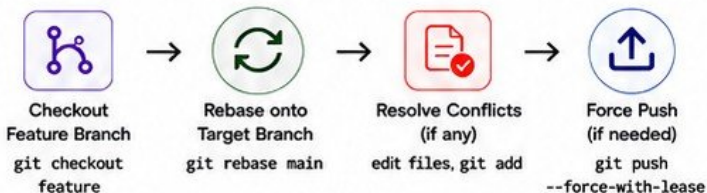
- White circles: common history
- Blue: main branch commits
- Green: feature branch commits
- Purple: merge commit



Git finds a common ancestor, compares changes, and creates a merge commit that brings both branches together.

4 7.4 REBASE WORKFLOW

Reapply commits from one branch on top of another.



5 7.5 INTERACTIVE REBASE

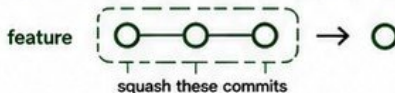
Edit, reorder, squash, or drop commits.

```
# Start interactive rebase (last 4 commits)
git rebase -i HEAD~4

# Common commands in interactive rebase
pick      Keep the commit
reword    Edit commit message
edit      Stop to amend the commit
squash    Combine with previous commit
drop      Remove the commit
```

6 7.6 SQUASHING COMMITS

Combine multiple commits into one clean commit.



```
# Interactive rebase
git rebase -i HEAD~4
# Mark commits as 'squash' (s)
# Save and exit
# Edit the final commit message
```

7 7.7 CHOOSING MERGE VS REBASE

USE MERGE WHEN...

- ✓ You want to preserve full history
- ✓ You're working on shared/public branches
- ✓ You value transparency of work
- ✓ Team policy requires merge
- ✓ Multiple developers collaborate on the branch

USE REBASE WHEN...

- ✓ You want a clean, linear history
- ✓ You're working on a private feature branch
- ✓ You want to remove WIP/extra commits
- ✓ You are the only one working on the branch
- ✓ Preparing for clean PR

8 7.8 SUMMARY



Merge = Safe, History preserved



Rebase = Clean, Linear history



Know your team's policy



Communicate before rewriting history



Always backup before force push

QUICK REFERENCE COMMANDS

```
git merge <branch>      # Merge branch into current branch
git merge --no-ff <branch> # Create merge commit even if fast-forward possible

git rebase <branch>      # Rebase current branch onto <branch>
git rebase -i HEAD~n    # Interactive rebase last n commits
git push --force-with-lease # Safe force push after rebase
git log --graph --oneline --all # Visualize history
```

BEST PRACTICES

- ✓ Keep commits small and meaningful.
- ✓ Pull latest changes before merging or rebasing.
- ✓ Resolve conflicts carefully and test your code.
- ✓ Never rebase commits that others have pulled.
- ✓ Use --force-with-lease instead of --force.
- ✓ Write good commit messages.



WARNING

Rewriting history can be dangerous on shared branches.

Always use rebase and force push with caution.





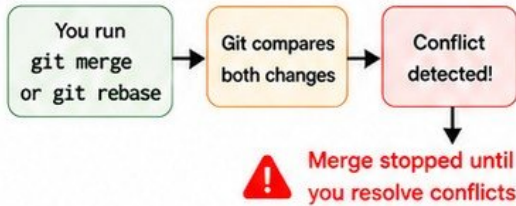
8. RESOLVING MERGE CONFLICTS



Conflicts happen. How you resolve them determines the quality of your code and history.

1 8.1 CONFLICT DETECTION

Git detects conflicts when the same lines are changed differently in two branches.



EXAMPLE (MERGE CONFLICT)

```

$ git checkout main
$ git merge feature/login
Auto-merging app.js
CONFLICT (content): Merge conflict in app.js
Automatic merge failed; fix conflicts and
then commit the result.
  
```



KEY POINT

Git will not complete the merge or rebase until conflicts are resolved and recorded.

2 8.2 CONFLICT MARKERS

Git adds special markers in the file to show conflicting changes.

```

// app.js
<<<<<< HEAD (current branch - main)
console.log("Welcome, User!");
=====
console.log("Welcome, Registered User!");
>>>>>> feature/login (incoming branch)
  
```

```

<<<<<< Start of current branch changes
===== Separator
>>>>>> End of incoming branch changes
  
```

3 8.3 MANUAL RESOLUTION

Edit the file, remove markers, keep the correct content, then stage and commit.

```

// app.js (resolved)
console.log("Welcome, Registered User!");
  
```

STEPS

- 1 Open the conflicted file(s)
- 2 Understand both changes
- 3 Edit and keep the desired content
- 4 Remove all conflict markers
- 5 Save the file
- 6 Stage the resolved file
`git add app.js`
- 7 Commit the resolution
`git commit -m "Resolve merge conflict in app.js"`

4 8.4 MERGE TOOLS

Use visual tools to compare and merge changes easily.

TOOL	PLATFORM
VS Code	Windows, macOS, Linux
GitKraken	Windows, macOS, Linux
SourceTree	Windows, macOS
JetBrains IDEA / PyCharm	Windows, macOS, Linux
Meld	Linux
Beyond Compare	Windows, macOS, Linux



TIP

Tools highlight differences side-by-side and let you accept changes with one click.

5 8.5 REBASE CONFLICTS

Rebase applies commits one by one. Conflicts can occur at any step.

```

$ git rebase main
Auto-merging app.js
CONFLICT (content): Merge conflict in app.js
error: could not apply alb2c3d... Add login
Resolve all conflicts manually, mark them as
resolved with "git add/rm <conflicted_files>",
then run "git rebase --continue".
  
```

RESUME AFTER RESOLUTION

```

$ git add <file>
$ git rebase --continue

$ git rebase --skip      # or skip
$ git rebase --abort    # or abort
  
```

★ REMEMBER

- ✓ Conflicts are normal.
- ✓ Good teams resolve them cleanly.
- ✓ Understand both sides before deciding.
- ✓ Clarity now prevents bugs later.

6 8.6 CONFLICT PREVENTION

Reduce conflicts before they happen.

- ✓ Pull latest changes frequently
`git pull --rebase`
- ✓ Communicate with your team
- ✓ Keep changes small and focused
- ✓ Avoid editing the same lines
- ✓ Use feature branches
- ✓ Run tests before merging
- ✓ Review code early and often
- ✓ Prefer rebase for clean history (on private branches)

</> GOOD COMMIT MESSAGE EXAMPLE

```

git commit -m "Resolve merge conflict in app.js:
use Registered User message and keep validation"
  
```

7 8.7 PRODUCTION RECOMMENDATIONS

Best practices for safe and clean collaboration.

- Always back up before major merges
`git branch backup/$(date +%F)`
- Resolve conflicts immediately, don't postpone
- Never commit conflict markers
- Test after resolving conflicts
- Use `pull --rebase` to minimize unnecessary merges
- Document important resolutions in commit messages



WARNING

Never force push shared branches after conflict resolution. It can overwrite teammates' work.

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



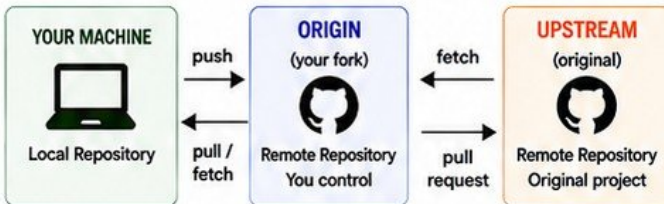
9. WORKING WITH REMOTE REPOSITORIES



Collaborate effectively by working with remote repositories.

1 9.1 ORIGIN AND UPSTREAM

Most projects use "origin" for your fork and "upstream" for the original repository.



You push to origin, but you fetch from upstream.

CHECK YOUR REMOTES

```
$ git remote -v
origin    https://github.com/you/repo.git (fetch)
origin    https://github.com/you/repo.git (push)
upstream  https://github.com/org/repo.git (fetch)
upstream  https://github.com/org/repo.git (push)
```

ADD REMOTE EXAMPLE

```
# Add upstream remote
$ git remote add upstream https://github.com/org/repo.git

# Verify
$ git remote -v
```

2 9.2 GIT FETCH



Downloads new data from remote repositories but does not change your working files.

```
$ git fetch origin
# or fetch from upstream
$ git fetch upstream
```

KEY POINTS

- Updates remote-tracking branches (e.g., origin/main)
- Safe: does not modify your work
- Use first to see what changed

3 9.3 GIT PULL



Fetches new changes and integrates them into your current branch.

```
$ git pull origin main
# or from upstream
$ git pull upstream main
```

KEY POINTS

- = git fetch + git merge
- May create a merge commit
- Ensure your work is clean before pulling

4 9.4 GIT PUSH



Sends your commits to the remote repository.

```
$ git push origin main
# push a new branch
$ git push -u origin feature/login
```

KEY POINTS

- Pushes to your remote (origin)
- Use -u to set upstream tracking
- May be rejected if remote has new changes

5 9.5 REMOTE TRACKING BRANCHES



Local references to the state of branches on a remote.

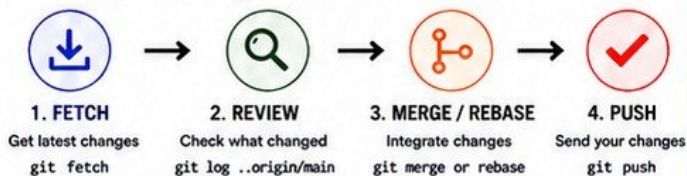
```
$ git branch -r
origin/HEAD -> origin/main
origin/main
origin/dev
upstream/main
```

KEY POINTS

- Starts with remote name (origin/, upstream/)
- Updated by git fetch
- Read-only references

6 9.6 SYNCHRONIZATION WORKFLOW

Keep your repository up to date.



PRO TIP

Always fetch and review before merging or rebasing. It prevents surprises and conflicts.

7 9.7 COLLABORATION WORKFLOW (EXAMPLE)

- 1 Clone the repository
- 2 Create a feature branch
- 3 Make changes and commit
- 4 Push your branch
- 5 Create a Pull Request
- 6 Address feedback
- 7 Merge and sync main

```
git clone <url>
git checkout -b feature/login
git add . && git commit -m "..."
git push -u origin feature/login
Open PR on GitHub
Commit and push updates
git pull origin main
```

8 COMMON REMOTE COMMANDS

```
$ git remote -v          # List remotes
$ git remote add <name> <url> # Add remote
$ git remote remove <name>    # Remove remote
$ git fetch <remote>         # Fetch from remote
$ git pull <remote> <branch> # Pull from remote
$ git push <remote> <branch> # Push to remote
$ git push -u <remote> <branch> # Push and set upstream
$ git branch -r             # List remote branches
$ git remote show <remote>  # Show remote info
```

9 BEST PRACTICES

- ✓ Fetch often, pull intentionally.
- ✓ Never push directly to main.
- ✓ Use feature branches.
- ✓ Keep commits small and focused.
- ✓ Write clear commit messages.
- ✓ Review before merging.
- ✓ Synchronize before starting work.
- ✓ Communicate with your team.

10 TROUBLESHOOTING TIPS

- ⚠ **Push rejected?**
Pull first and resolve conflicts.
- ⚠ **Non-fast-forward error?**
Remote has new commits.
- ⚠ **Lost commits after pull?**
Check: git reflog
- ⚠ **Wrong remote?**
Check: git remote -v



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

VERIQTA SOCIAL



10. GITHUB ESSENTIALS

Fundamental concepts and best practices for using GitHub effectively.



1 10.1 REPOSITORY CREATION

Create a new repository on GitHub.

- 1 Sign in to GitHub
- 2 Click "New" or the + icon → New repository
- 3 Enter repository name
Use lowercase and hyphens (e.g., my-awesome-project)
- 4 Add description (recommended)
- 5 Choose visibility
- 6 Initialize repository (optional)
 - Add a README file
 - Add .gitignore
 - Choose a license
- 7 Click "Create repository"

VISIBILITY OPTIONS



Public
Anyone on the internet can see and fork.



Private
Only you and collaborators can see.



Internal (GitHub Team/Org)
Visible to members of your organization.



TIP
Start with a README. It makes your project discoverable and professional.

2 10.2 REPOSITORY SETTINGS

Configure important settings for your repository.

	General	Repository name, description, website, default branch.
	Collaborators	Add, remove, and manage access permissions.
	Branches	Set default branch, branch protection rules.
	Webhooks	Integrate with external services and automation.
	Actions & Secrets	Manage GitHub Actions permissions and repository secrets.
	Advanced	Transfer ownership, archive, delete repository.

3 10.3 REPOSITORY VISIBILITY

Choose the right visibility for your project.

VISIBILITY	BEST FOR	CONSIDERATIONS
Public	Open source projects, portfolio, learning	Great for visibility and collaboration.
Private	Proprietary code, client projects	Only invited collaborators can access.
Internal	Company projects (shared within org)	Accessible to all org members.



You can change visibility at any time (except public repos with forks).

4 10.4 README FILES

The README is the front page of your repository.

```
# Project Title
Short description of your project.

## Table of Contents
- [Overview](#overview)
- [Installation](#installation)
- [Usage](#usage)
- [Contributing](#contributing)
- [License](#license)

## Overview
Explain what your project does.

## Installation
Steps to install dependencies.

## Usage
Examples and usage instructions.

## Contributing
How others can contribute.

## License
Link to your license.
```

BEST PRACTICES

- ✓ Use clear project title and description.
- ✓ Add table of contents for large docs.
- ✓ Include installation and usage steps.
- ✓ Add screenshots or demos.
- ✓ Explain contribution guidelines.
- ✓ Link to license.
- ✓ Keep it updated!

5 10.5 LICENSE

Add a license to define how others can use your code.

POPULAR LICENSES



MIT
Permissive. Allows reuse with attribution.



Apache 2.0
Permissive. Includes patent grant and protections.



GPL v3
Copyleft. Derivatives must be open source.



BSD 3-Clause
Permissive. Similar to MIT.

HOW TO ADD

Option 1: GitHub UI

1. Go to Settings → General
2. Scroll to "Features"
3. Click "Choose a license"
4. Select a license → Commit

Option 2: Command line

```
$ echo "MIT License" > LICENSE
$ git add LICENSE
$ git commit -m "Add MIT license"
$ git push
```



LICENSE is a legal document. Choose what fits your project goals.

6 10.6 .GITIGNORE

Keep your repository clean by ignoring unnecessary files.

EXAMPLE: PYTHON .GITIGNORE

```
# Byte-compiled / optimized
__pycache__/
*.py[co]
*.pyc

# Virtual environments
.venv/
env/
envs/

# Logs
*.log

# OS files
.DS_Store
Thumbs.db

# IDE files
.vscode/
.idea/
```

BEST PRACTICES

- ✓ Add .gitignore before your first commit.
- ✓ Use templates from GitHub's .gitignore repository.
- ✓ Review it periodically.
- ✓ Ignore build, logs, secrets, and system files.
- ✓ Never ignore important source files.

7 10.7 REPOSITORY ORGANIZATION

A well-organized repository is easy to understand and maintain.

PROJECT STRUCTURE EXAMPLE

```
my-awesome-project/
├── README.md
├── LICENSE
├── .gitignore
├── docs/
│   └── architecture.md
├── src/
│   ├── app.py
│   └── utils.py
├── tests/
│   └── test_app.py
└── requirements.txt
```

ORGANIZATION TIPS

- ✓ Keep root directory clean.
- ✓ Group related files in folders.
- ✓ Use meaningful folder names.
- ✓ Separate source, tests, docs.
- ✓ Add docs/ for documentation.
- ✓ Use consistent naming conventions.
- ✓ Keep configuration files in the root.

USE GITHUB FEATURES



Issues
Track bugs, feature requests, tasks.



Projects
Board views for planning and tracking.



Labels
Categorize and prioritize work.



Milestones
Set goals and release targets.



PRO TIP

A clean repo structure + good documentation + clear licensing = a professional open source project.



REMEMBER

- ✓ Initialize with README, LICENSE, .gitignore.
- ✓ Keep documentation clear and updated.
- ✓ Protect important branches.
- ✓ Collaborate transparently.
- ✓ Quality repos build trust!



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



11. PULL REQUESTS AND CODE REVIEWS



High quality code is built through great reviews and clear collaboration.

1 11.1 PULL REQUEST LIFECYCLE

The typical lifecycle of a pull request.



TIP: Keep PRs small, focused, and easy to review.
Small PRs = faster reviews and fewer defects.

2 11.2 REVIEW PROCESS

A structured approach to code reviews.

- 1 Understand the purpose of the change
- 2 Review the code for correctness and quality
- 3 Check for performance, security, and edge cases
- 4 Verify tests and documentation
- 5 Provide clear, respectful feedback
- 6 Approve or request changes
- 7 Re-verify after changes are made

REVIEW CHECKLIST

- ✓ Functionality correct?
- ✓ Edge cases handled?
- ✓ Performance considered?
- ✓ Security reviewed?
- ✓ Tests added/updated?
- ✓ Code is readable?
- ✓ Documentation updated?

3 11.3 REVIEW COMMENTS

Use comments to communicate clearly.

reviewer Reviewer 2 hours ago ...

Great work overall! The logic looks good. Consider extracting this block into a function to improve readability.

```
42 if (user.isActive()) {
43   sendEmail(user);
44   logActivity(user);
45 }
```

Add a reply...



TIP: Be specific, kind, and constructive.
Good comments help everyone learn.

4 11.4 SUGGESTED CHANGES

Suggest improvements directly in the code.

Suggested change

```
42 if (user.isActive()) {
43   - sendEmail(user);
44   - logActivity(user);
45 }
↓
42 if (user.isActive()) {
43   + notifyUser(user);
44   + logUserActivity(user);
45 }
```



TIP: Use suggested changes to make it easy for authors to apply feedback.

5 11.5 APPROVAL WORKFLOW

Define who can approve and when.

	REVIEWERS At least one required reviewer must approve.
	CODE OWNER Code owners' approval may be required.
	AUTOMATION All checks (CI, tests, scans) must pass.
	STATUS When all requirements are met, PR is ready to merge.



TIP: Establish clear rules for who can approve and under what conditions.

6 11.6 MERGE REQUIREMENTS

Protect your codebase with rules and checks.

REQUIREMENT	DESCRIPTION
Required Reviews	Minimum number of reviewers must approve.
Status Checks	All CI/CD checks must pass (build, test, lint, security).
Branch Protection	Restrict direct pushes to main/default branch.
Up to Date	Require branches to be up to date before merging.
Signed Commits	Require signed commits (recommended).

7 11.7 ENTERPRISE REVIEW STANDARDS

Follow industry best practices for consistent, high-quality reviews.

CODE QUALITY
✓ Follow coding standards and conventions ✓ Write clear, maintainable code ✓ Avoid duplication and complexity ✓ Handle errors and edge cases ✓ Optimize for readability and performance
SECURITY & COMPLIANCE
🔒 Validate input and output 🔒 Avoid hardcoded secrets 🔒 Check dependencies for vulnerabilities 🔒 Follow compliance requirements

COMMUNICATION
✓ Be respectful and professional ✓ Ask questions when unsure ✓ Explain the why, not just the what ✓ Acknowledge good work
DOCUMENTATION & TESTING
✓ Update documentation ✓ Add or update tests ✓ Ensure tests are meaningful ✓ Keep README and docs current



BEST PRACTICES

- ✓ Keep pull requests small and focused.
- ✓ Write a clear PR description.
- ✓ Link related issues.
- ✓ Review regularly and promptly.



PRO TIP

Great reviews don't just find problems, they make the codebase and the team better.

REVIEW → IMPROVE → MERGE → DELIVER



REMEMBER

- ✓ Quality today prevents bugs tomorrow.
- ✓ Everyone is a reviewer. Everyone is a learner.
- ✓ Code review is a quality multiplier.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

VERIQTA SOCIAL



12. GITHUB COLLABORATION



Work together effectively as a team and ship better software.

1 TEAMS

Organize people into teams for efficient collaboration.



- ✓ Create teams for different projects or domains
- ✓ Add members to teams
- ✓ Manage team visibility (private or internal)
- ✓ Sync with your organization or directory (SSO)
- ✓ Use teams to manage access at scale



TIP: Use teams to simplify access management and keep security strong.

2 REPOSITORY PERMISSIONS

Control who can access and what they can do.

ROLE	PERMISSIONS
READ	View, clone, and open issues and pull requests.
TRIAGE	Manage issues and PRs (without write access).
WRITE	Push to the repository and create branches.
MAINTAIN	Manage settings, labels, webhooks, and branches.
ADMIN	Full access including delete and manage access.



TIP: Grant the minimum access necessary for each team member.

3 CODEOWNERS

Automatically request reviews from the right people.

Example CODEOWNERS file

```
# Global owners
* @team/platform

# Specific directories
/src/ @team/backend
/frontend/ @team/web
/infra/ @team/devops

# Specific files
Dockerfile @team/devops
package.json @team/backend
```



TIP: CODEOWNERS ensures the right experts review the right changes.

4 BRANCH PROTECTION

Protect important branches from accidental or unwanted changes.



- ✓ Require pull requests before merging
- ✓ Require approvals before merging
- ✓ Require status checks to pass
- ✓ Prevent force pushes
- ✓ Restrict who can push or merge
- ✓ Require conversation resolution
- ✓ Lock branch (admin only)



TIP: Protect main, develop, and release branches to keep your codebase safe.

5 REQUIRED REVIEWERS

Ensure quality by requiring reviews before changes are merged.

- ✓ Require at least one or more reviewers
- ✓ Specify teams or individual reviewers
- ✓ Combine with CODEOWNERS for automated requests
- ✓ Approvals are required before merge
- ✓ Re-review after new commits (if required)



TIP: More reviewers = better code, less risk, and shared knowledge.

6 DISCUSSIONS

Have conversations around ideas, plans, and feedback.



- ✓ Start discussions for ideas or announcements
- ✓ Get feedback from your team or community
- ✓ Use categories to organize topics
- ✓ Convert discussions into issues or PRs
- ✓ Keep everyone aligned and informed



TIP: Use Discussions for early feedback before creating issues or PRs.

7 PROJECT COLLABORATION

Use GitHub Projects to plan, track, and deliver work together.



PLANNING

Create boards to plan features, roadmaps, and tasks.



TRACKING

Track progress with customizable views and fields.



ASSIGNMENTS

Assign tasks to team members and set deadlines.



AUTOMATION

Automate workflows with rules and integrations.



INSIGHTS

Get insights with charts, filters, and real-time updates.



TIP: GitHub Projects keeps your team organized and your work moving forward.

8 COLLABORATION BEST PRACTICES

- ✓ Communicate early and often.
- ✓ Keep pull requests small and focused.
- ✓ Write clear descriptions and context.
- ✓ Review with respect and be constructive.
- ✓ Resolve conversations before merging.
- ✓ Use templates for issues and PRs.
- ✓ Celebrate wins and learn from mistakes.
- ✓ Keep documentation up to date.



TIP: Great collaboration builds better software and stronger teams.

9 COLLABORATION WORKFLOW (EXAMPLE)



1. PLAN
Discuss and plan in Discussions or Projects.

2. CREATE BRANCH
Create a branch for the work.

3. DEVELOP
Make changes and commit regularly.

4. OPEN PR
Open a Pull Request and request reviews.

5. REVIEW
Reviewers check, comment, and approve.

6. MERGE
Merge after all requirements are met.

7. DELIVER
Ship and iterate continuously.

★ KEY REMINDER

Collaboration is more than tools. It's about people, communication, and a shared commitment to quality and success.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



13. GITHUB ISSUES AND PROJECT MANAGEMENT



Plan work, track progress, and deliver quality software.

1 ISSUES

Track tasks, bugs, features, and improvements.



- ✓ Create issues to capture work
- ✓ Describe the problem or task
- ✓ Assign owners and watchers
- ✓ Link related issues and PRs
- ✓ Track status and progress
- ✓ Close issues when resolved

TIP: Good issues are clear, concise, and actionable.

2 LABELS

Categorize and prioritize work with labels.

- bug** Something isn't working
- feature** New functionality
- enhancement** Improve existing feature
- documentation** Docs and guides
- priority: high** High priority work
- help wanted** Needs community help

TIP: Use labels consistently across the repository.

3 MILESTONES

Group issues into goals and track progress.



- ✓ Create milestones for releases or major goals
- ✓ Set due dates and descriptions
- ✓ Assign issues to milestones
- ✓ Monitor completion progress
- ✓ Celebrate achievements when completed

TIP: Milestones keep your team focused and aligned.

4 PROJECTS

Organize and manage work in a visual way.



- ✓ Create projects for initiatives or products
- ✓ Add issues, PRs, and notes
- ✓ Filter, group, and sort work
- ✓ Track progress in real time
- ✓ Share with your team

TIP: Projects provide visibility across all active work.

5 KANBAN BOARDS

Visualize workflow and limit work in progress.

TO DO	IN PROGRESS	IN REVIEW	DONE
Issue #101 Login bug	Issue #104 API endpoint	PR #106 Refactor code	Issue #98 Fix crash
Issue #102 Add validation	Issue #105 UI improvement	Issue #107 Write tests	PR #99 Improve performance
Issue #103 Update docs	+ Add issue	+ Add issue	+ Add issue
+ Add issue			

TIP: Limit work in progress to improve flow and reduce cycle time.

6 ISSUE TEMPLATES

Standardize how issues are reported.

Example Bug Report Template

****Describe the bug****
A clear and concise description of the bug.

****Steps to reproduce****

1. Go to '...'
2. Click on '...'
3. See error

****Expected behavior****
What you expected to happen.

****Screenshots****
Attach screenshots if applicable.

****Environment****

- OS:
- Browser / Version:
- Additional context:

TIP: Templates save time and ensure complete information.

7 SPRINT PLANNING

Plan work, set goals, and deliver iteratively.

	DEFINE GOALS Set clear sprint goals and success criteria.
	SELECT WORK Choose issues from backlog for the sprint.
	ESTIMATE Estimate effort and size of selected work.
	PLAN CAPACITY Match work to team capacity and availability.
	TRACK & ADAPT Monitor progress daily and adjust as needed.

TIP: Short sprints + regular reviews lead to consistent delivery.

8 ISSUE LIFECYCLE From creation to completion.



1. OPEN

Issue is created and recorded.



2. ASSIGNED

Owner is assigned to the issue.



3. IN PROGRESS

Work begins and updates are added.



4. RESOLVED

Work is completed and marked resolved.



5. CLOSED

Issue is reviewed and closed.

★ BEST PRACTICES

- ✓ Write clear and detailed issue descriptions.
- ✓ Use labels, milestones, and projects.
- ✓ Keep issues and projects up to date.
- ✓ Link related issues and pull requests.
- ✓ Communicate early and often.
- ✓ Review and close completed work.



14. GITHUB ACTIONS FUNDAMENTALS



Automate your workflows. Build better. Ship faster.

1 WORKFLOW FILES

Define automation in YAML workflow files.



- ✓ Stored in `.github/workflows/`
- ✓ Use YAML syntax
- ✓ Version controlled with code
- ✓ Reusable and shareable

TIP: One workflow file can run many jobs.

2 EVENTS

Workflows run when specific events occur.



push Code pushed to repository.



pull_request PR opened, synchronized, or closed



issues Issue opened or closed



schedule On a defined cron schedule



workflow_dispatch Manually triggered

TIP: Choose the right event to trigger your workflow.

3 JOBS

A workflow runs one or more jobs, which run in parallel by default.

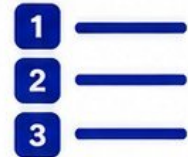


- ✓ Runs on a runner
- ✓ Can depend on other jobs
- ✓ Use needs to set dependencies
- ✓ Jobs can produce outputs

TIP: Use jobs to separate tasks and control order.

4 STEPS

Steps are individual tasks within a job.



- ✓ Run commands or actions
- ✓ Each step runs in sequence
- ✓ Use actions from marketplace
- ✓ Steps can use outputs

TIP: Keep steps small, focused, and reusable.

5 RUNNERS

Runners execute your jobs.

GITHUB-HOSTED



- Managed by GitHub
- Ubuntu, Windows, macOS
- Easy to use
- Free for public repos

SELF-HOSTED



- Run on your own infrastructure
- Full control
- Better for private repos and large workloads

TIP: Use self-hosted runners for specialized environments and scalability.

6 WORKFLOW EXECUTION

How a workflow runs from start to finish.



Event Triggered



Workflow Loaded



Jobs Created



Steps Executed



Workflow Completed

EXECUTION STATES

SUCCESS

Completed successfully

FAILURE

Completed with errors

CANCELLED

Manually cancelled

SKIPPED

Condition not met

IN PROGRESS

Currently running

TIP: Check the Actions tab to monitor runs and debug failures.

7 CI BASICS

Continuous Integration with GitHub Actions.



- ✓ Automate build, test, and lint
- ✓ Catch issues early
- ✓ Ensure code quality
- ✓ Faster feedback to developers
- ✓ Consistent and repeatable

TIP: CI is the foundation of DevOps excellence.

EXAMPLE: SIMPLE CI WORKFLOW

`.github/workflows/ci.yml`

```
1 name: Simple CI
2 on: [push, pull_request]
3
4 jobs:
5   build-and-test:
6     runs-on: ubuntu-latest
7     steps:
8       - name: Checkout code
9         uses: actions/checkout@v4
10      - name: Set up Node.js
11        uses: actions/setup-node@v4
12        with:
13          node-version: '18'
14      - name: Install dependencies
15        run: npm ci
16      - name: Run tests
17        run: npm test
18      - name: Lint code
19        run: npm run lint
```

WHAT THIS WORKFLOW DOES

- 1 Triggers on push or PR
- 2 Runs on latest Ubuntu
- 3 Checks out your code
- 4 Sets up Node.js
- 5 Installs dependencies
- 6 Runs tests
- 7 Runs linter
- 8 Fails the job if any step fails

BEST PRACTICES

- ✓ Keep workflows simple and readable
Use clear names and comments.
- ✓ Use reusable workflows and composite actions
Avoid duplication and save time.
- ✓ Pin action versions
Use specific versions (e.g., @v4) for stability.
- ✓ Use caching to speed up builds
Cache dependencies and build artifacts.
- ✓ Fail fast and alert early
Detect problems before they reach production.
- ✓ Secure your workflows
Use least privilege and avoid hardcoding secrets.
- ✓ Monitor and improve
Review logs and optimize continuously.

★ QUICK REMINDER

- ✓ Workflows = Automation
- ✓ Jobs = Tasks
- ✓ Steps = Actions
- ✓ Runners = Execution
- ✓ CI = Quality and speed
- ✓ Automate everything repeatable!



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

VERIQTA SOCIAL



15. PRODUCTION CI/CD WITH GITHUB ACTIONS



Automate. Test. Secure. Build. Deploy. Repeat.

1 BUILD PIPELINES

Compile, package, and prepare your application.



- ✓ Checkout code
- ✓ Set up environment
- ✓ Install dependencies
- ✓ Compile / build application
- ✓ Package build artifacts
- ✓ Upload artifacts for later jobs

TIP: Keep builds fast, reliable, and repeatable.

2 AUTOMATED TESTING

Ensure quality with automated tests at every stage.



- ✓ Unit tests
- ✓ Integration tests
- ✓ End-to-end tests
- ✓ Code coverage reports
- ✓ Test result reporting
- ✓ Fail fast on test failures

TIP: Test early, test often, prevent regressions.

3 SECURITY SCANNING

Find and fix vulnerabilities automatically.



- ✓ Dependency vulnerability scan
- ✓ Code quality analysis
- ✓ Secret detection
- ✓ Container image scan
- ✓ SAST (Static Analysis)
- ✓ Fail build on critical issues

TIP: Shift security left and make it part of CI.

4 DOCKER BUILDS

Build and publish container images consistently.



- ✓ Build Docker image
- ✓ Tag with version / SHA
- ✓ Run container tests
- ✓ Push to container registry
- ✓ Use build cache for speed
- ✓ Scan image for vulnerabilities

TIP: Build once, run anywhere with containers.

5 DEPLOYMENT AUTOMATION

Deploy automatically with confidence.



- ✓ Deploy to dev / staging / prod
- ✓ Use infrastructure as code
- ✓ Blue/Green deployments
- ✓ Rolling deployments
- ✓ Health checks after deploy
- ✓ Automatic rollback on failure

TIP: Automate deployments, reduce manual errors.

6 ENVIRONMENT APPROVALS

Add human checkpoints for controlled releases.



- ✓ Require approval for staging / production
- ✓ Environment protection rules
- ✓ Manual approval gates
- ✓ Audit trail for approvals
- ✓ Role-based access control (RBAC)
- ✓ Prevent unauthorized deployments

TIP: Protect production with approvals and rules.

7 RELEASE WORKFLOW

Create consistent, repeatable releases.



- ✓ Create GitHub Release
- ✓ Auto-generate release notes
- ✓ Version tagging
- ✓ Publish artifacts / packages
- ✓ Notify teams and stakeholders
- ✓ Track deployment status

TIP: Make releases predictable and traceable.

8 END-TO-END CI/CD PIPELINE EXAMPLE



9 EXAMPLE: SIMPLE WORKFLOW STRUCTURE

```
1 name: CI/CD Pipeline
2 on:
3   push:
4     branches: [ main ]
5   pull-request:
6     branches: [ main ]
7 jobs:
8   build-and-test:
9     runs-on: ubuntu-latest
10    steps:
11      - uses: actions/checkout@v4
12      - uses: actions/setup-node@v4
13        with:
14          node-version: '18'
15      - run: npm ci
16      - run: npm test
17      - run: npm run build
```



KEY REMINDER:

A great CI/CD pipeline delivers software faster, safer, and with higher quality.

10 BEST PRACTICES

- ✓ Keep workflows simple and modular
Use reusable workflows and composite actions.
- ✓ Fail fast
Stop early when something breaks.
- ✓ Use caching
Cache dependencies and build artifacts.
- ✓ Secure everything
Use secrets, least privilege, and OIDC.
- ✓ Version everything
Pin action versions and dependencies.
- ✓ Monitor and improve
Track pipeline metrics and optimize.



CI/CD PRINCIPLE:

Automate everything that can be automated. Validate everything. Deploy with confidence.

11 SUCCESS METRICS

✓ Build Success Rate	> 95%
🕒 Pipeline Duration	< 10 minutes
🛡️ Deployment Frequency	Multiple times per day
⚠️ Change Failure Rate	< 5%
👍 Mean Time to Recover	< 1 hour



GOAL:

Faster delivery. Higher quality. Happier users. Better teams.



16. GIT SECURITY BEST PRACTICES



Secure your code. Protect your data. Build with confidence.

1 SSH KEYS

Use SSH keys for secure authentication.



- ✓ Generate a strong SSH key using ed25519
- ✓ Add public key to GitHub
- ✓ Use SSH URLs, not HTTPS
- ✓ Protect private key with a passphrase
- ✓ Rotate keys regularly

 **TIP:** Never share your private SSH key.

2 PERSONAL ACCESS TOKENS (PAT)

Use tokens instead of passwords for HTTPS access.



- ✓ Create fine-grained tokens
- ✓ Grant minimum required scopes
- ✓ Set expiration dates
- ✓ Store tokens securely
- ✓ Revoke unused tokens

 **TIP:** Treat tokens like passwords. Keep them safe.

3 SECRET MANAGEMENT

Protect secrets and sensitive information.



- ✓ Use GitHub Secrets for Actions
- ✓ Never commit secrets to code
- ✓ Use environment secrets
- ✓ Use secret scanning
- ✓ Rotate secrets regularly

 **TIP:** If it's sensitive, don't commit it.

4 COMMIT SIGNING

Verify the integrity and authenticity of commits.



- ✓ Enable GPG or SSH signing
- ✓ Sign all commits
- ✓ Verify signatures on GitHub
- ✓ Use verified badges
- ✓ Prevent unsigned commits

 **TIP:** Signed commits build trust and accountability.

5 REPOSITORY SECURITY

Lock down your repositories.



- ✓ Make repos private by default
- ✓ Enable vulnerability alerts
- ✓ Enable Dependabot
- ✓ Restrict who can push
- ✓ Audit repository access
- ✓ Enable secret scanning

 **TIP:** Review access and permissions regularly.

6 BRANCH PROTECTION

Prevent unwanted or unsafe changes.



- ✓ Require pull requests
- ✓ Require approvals
- ✓ Require status checks to pass
- ✓ Restrict pushes to protected branches
- ✓ Require linear history
- ✓ Prevent force pushes
- ✓ Lock important branches

 **TIP:** Protect main, develop, and release branches at all times.

7 SUPPLY CHAIN SECURITY

Secure your dependencies and workflows.



- ✓ Pin dependency versions
- ✓ Use trusted sources
- ✓ Scan dependencies for vulnerabilities
- ✓ Verify GitHub Actions and third-party actions
- ✓ Use OpenID Connect (OIDC)
- ✓ Audit workflow permissions
- ✓ Follow least privilege principle

 **TIP:** Secure the full chain—code, dependencies, actions, and infrastructure.



KEY REMINDER

Security is not a feature,
it's a foundation.



Secure Access



Protect Secrets



Verify Integrity



Control Access



Stay Vigilant

VERIQTA SOCIAL



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



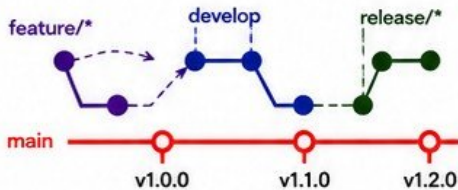
17. ENTERPRISE GIT WORKFLOWS



Choose the right workflow. Ship quality. Scale with confidence.

1 GITFLOW

Ideal for projects with scheduled releases.



- ✓ Feature branches from develop
- ✓ Release branches for stabilization
- ✓ Hotfix branches from main
- ✓ Merge back to develop and main
- ✓ Best for large teams and complex releases



TIP: Use GitFlow when you need structured releases and hotfixes.

2 GITHUB FLOW

Ideal for continuous delivery environments.



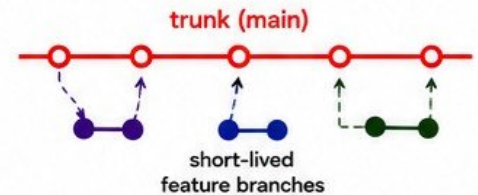
- ✓ Create feature branch from main
- ✓ Commit and push changes
- ✓ Open Pull Request
- ✓ Review and merge to main
- ✓ Deploy immediately
- ✓ Simple, fast, and team-friendly



TIP: Use GitHub Flow for continuous delivery and fast iterations.

3 TRUNK-BASED DEVELOPMENT

Ideal for high-velocity, CI/CD teams.



- ✓ Work directly on trunk (main)
- ✓ Short-lived branches (hours/days)
- ✓ Feature flags for incomplete work
- ✓ Continuous integration
- ✓ Always deployable trunk
- ✓ Best for fast-paced teams



TIP: Keep trunk always releasable. Automate everything.

4 RELEASE MANAGEMENT

Plan, build, test, and release with control.



- ✓ Plan releases and define scope
- ✓ Use release branches or tags
- ✓ Run full test suite
- ✓ Update documentation
- ✓ Create release notes
- ✓ Tag release
- ✓ Deploy to production
- ✓ Monitor and validate



TIP: Use tags and release notes to communicate value clearly.

5 VERSIONING

Use clear versioning to track changes.



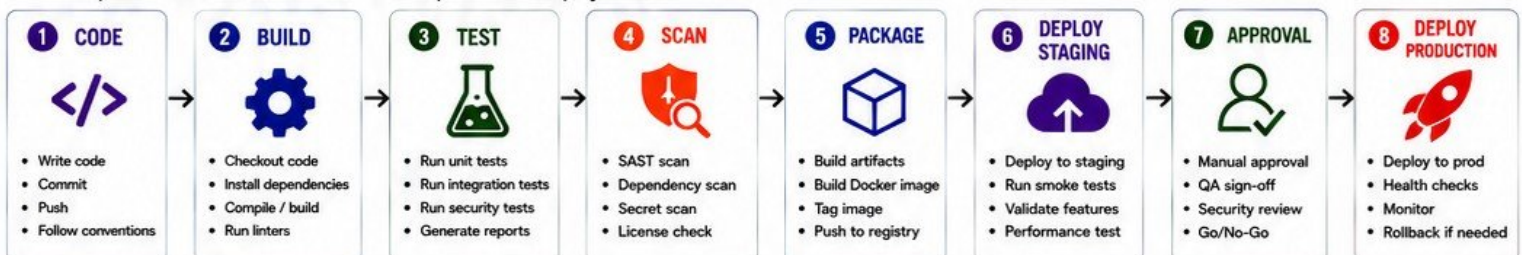
TIP: Follow SemVer and never reuse version numbers.

EXAMPLES

- ✓ 1.0.0 Initial release
- ✓ 1.1.0 New feature
- ✓ 1.1.1 Bug fix
- ✓ 2.0.0 Breaking change

6 PRODUCTION DEPLOYMENT WORKFLOW

A proven workflow for safe and reliable production deployments.



BEST PRACTICES

- ✓ Automate builds, tests, and deployments.
- ✓ Keep main branch always deployable.
- ✓ Protect production with approvals.
- ✓ Monitor, log, and improve continuously.
- ✓ Document and version everything.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

VERIQTA SOCIAL



18. GIT TROUBLESHOOTING



Diagnose problems. Recover quickly. Keep shipping.

1 DETACHED HEAD



You are not on a branch.
Commits may be lost.

- ✓ Happens when checking out a commit or tag
- ✓ Work is safe but not on a branch
- ✓ Create a branch to save work

WHAT TO DO

```
# Check status
git status

# Create branch from here
git switch -c my-branch

# Or go back to last branch
git switch -
```

TIP: Always create a branch before making changes.

2 RECOVERING DELETED BRANCHES



Branches are not always permanently lost.

- ✓ Git keeps references in reflog
- ✓ You can restore the branch
- ✓ Act quickly before GC runs

WHAT TO DO

```
# View reflog
git reflog

# Find the commit of deleted branch

# Restore branch
git branch recovered <commit>
```

TIP: Reflog is your best friend for recovery.

3 UNDOING COMMITS



Undo local commits safely without losing work.

- ✓ Soft reset keeps changes
- ✓ Mixed reset unstages changes
- ✓ Hard reset deletes changes

WHAT TO DO

```
# Undo last commit, keep changes
git reset --soft HEAD~1

# Undo last commit, unstage
git reset --mixed HEAD~1

# Undo last commit, delete
git reset --hard HEAD~1
```

TIP: Use --soft or --mixed unless you are sure.

4 RECOVERING LOST WORK



Lost changes can often be recovered.

- ✓ Check reflog
- ✓ Check stash
- ✓ Check fsck for dangling commits

WHAT TO DO

```
# Check reflog
git reflog

# Check stash list
git stash list

# Check dangling commits
git fsck --lost-found
```

TIP: Reflog and stash can save your work.

5 RESET VS REVERT



Understand the difference to choose the right tool.

RESET	REVERT
Moves HEAD and branch	Creates a new commit
Rewrites history	Preserves history
Use for local changes	Use for shared branches
Can lose work	Safe and traceable

EXAMPLES

```
git reset --soft HEAD~1
git reset --mixed HEAD~1
git reset --hard HEAD~1

git revert <commit>
git revert HEAD
```

TIP: Use revert on shared branches. Use reset on local branches.

6 STASH RECOVERY



Stashed changes can be recovered anytime.

- ✓ View stash list
- ✓ Apply or pop stash
- ✓ Recover dropped stash

WHAT TO DO

```
# List stashes
git stash list

# Apply stash
git stash apply stash@{0}

# Pop stash
git stash pop stash@{0}

# Recover dropped stash
git fsck --lost-found
```

TIP: Dropped stashes can still be recovered.

7 COMMON PRODUCTION ISSUES



- ✓ Accidental push to main
- ✓ Force push and lost history
- ✓ Merge conflicts in production
- ✓ Permission denied
- ✓ Large files in repository
- ✓ CI/CD pipeline failures
- ✓ Branch protection blocks push
- ✓ Detached HEAD in CI/CD
- ✓ Wrong environment deployed

HOW TO APPROACH

1. Stop and assess the impact
2. Check logs and error messages
3. Reproduce the issue locally
4. Identify the root cause
5. Fix and test thoroughly
6. Deploy safely and verify

TIP: Stay calm. Follow a process. Document and learn.

QUICK REFERENCE COMMANDS

COMMAND	PURPOSE	COMMAND	PURPOSE
git status	Check current status	git reflog	Show reference log
git log --oneline --graph --all	View commit history	git fsck --lost-found	Find lost objects
git branch -a	List all branches	git stash list	List stashes
git remote -v	View remotes	git stash show -p stash@{0}	Show stash changes
git fetch --all --prune	Update all references	git diff	Show unstaged changes
git clean -fd	Remove untracked files	git diff --staged	Show staged changes

BEST PRACTICES



- ✓ Commit small and often
- ✓ Pull before you push
- ✓ Use meaningful commit messages
- ✓ Protect main and release branches
- ✓ Review before merging
- ✓ Automate testing and checks
- ✓ Backup important repositories
- ✓ Document recovery steps



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta

VERIQTA SOCIAL



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



19. REAL PRODUCTION LABS



Practice real-world workflows. Build confidence. Ship like a pro.

1 FEATURE DEVELOPMENT WORKFLOW



- 1 Create feature branch
- 2 Develop and commit
- 3 Push to remote
- 4 Open Pull Request
- 5 Merge after review

EXAMPLE COMMANDS

```
# Create branch
git checkout -b feature/login
# Work and commit
git add .
git commit -m "Add login feature"
# Push branch
git push -u origin feature/login
```

TIP: Small, focused commits make reviews easier.

2 MULTI-DEVELOPER COLLABORATION



- 1 Multiple developers work in parallel
- 2 Share changes frequently
- 3 Keep branches up to date
- 4 Communicate and coordinate

EXAMPLE COMMANDS

```
# Sync with main
git checkout main
git pull origin main
# Update feature branch
git checkout feature/login
git rebase main
git push -f origin feature/login
```

TIP: Pull often. Rebase before creating PR.

3 PULL REQUEST REVIEW



- 1 Create Pull Request
- 2 Automated checks run
- 3 Code review by team
- 4 Address feedback
- 5 Merge when approved

EXAMPLE CHECKS

- ✓ Lint and format
- ✓ Unit tests
- ✓ Security scan
- ✓ Build success
- ✓ Code review approval

TIP: Reviews catch bugs before production.

4 CONFLICT RESOLUTION



- 1 Pull latest changes
- 2 Resolve conflicts
- 3 Test your changes
- 4 Commit and push

EXAMPLE COMMANDS

```
# Pull latest
git pull origin main
# Resolve conflicts in files
# Mark resolved
git add <file>
# Continue rebase
git rebase --continue
```

TIP: Communicate early to avoid long-lived conflicts.

5 PRODUCTION RELEASE SIMULATION



- 1 Prepare release branch
- 2 Update version
- 3 Run tests and validations
- 4 Merge to main
- 5 Tag the release
- 6 Deploy to production

EXAMPLE COMMANDS

```
git checkout -b release/v1.2.0
# Update version in files
git add .
git commit -m "chore: release v1.2.0"
git checkout main
git merge release/v1.2.0
git tag -a v1.2.0 -m "Release v1.2.0"
git push origin main --tags
```

TIP: Tag every release for traceability.

6 ROLLBACK WORKFLOW



- 1 Identify last stable release
- 2 Revert or reset to stable commit
- 3 Deploy the rollback
- 4 Verify application health
- 5 Communicate to team

EXAMPLE COMMANDS

```
# Option 1: Revert (safe)
git revert <bad-commit>
git push origin main
# Option 2: Reset (use with caution)
git reset --hard <stable-commit>
git push --force-with-lease
```

TIP: Revert is safer. Reset is faster but riskier.

7 INCIDENT RECOVERY USING GIT



- 1 Identify what changed
- 2 Find the breaking commit
- 3 Revert the change
- 4 Deploy the fix
- 5 Investigate and document
- 6 Prevent recurrence

EXAMPLE COMMANDS

```
# Find breaking commit
git log --oneline --graph
# Revert the commit
git revert <bad-commit>
# Push and deploy
git push origin main
```

TIP: Git history is your best incident response tool.

REAL-WORLD SCENARIOS

SCENARIO	DESCRIPTION	KEY COMMANDS
Feature Development	Build new features safely	checkout -b, add, commit, push
Team Collaboration	Multiple devs working together	pull, rebase, push, fetch
Code Review	Review and improve code quality	pull request, review, merge
Conflict Resolution	Fix conflicts between branches	pull, resolve, add, rebase --continue
Production Release	Release new version to production	merge, tag, push, deploy
Rollback	Revert to a stable version	revert, reset, push
Incident Recovery	Recover from production issues	log, revert, deploy, document

BEST PRACTICES



- ✓ Commit small and meaningful changes
- ✓ Pull and rebase frequently
- ✓ Use Pull Requests for all changes
- ✓ Resolve conflicts quickly
- ✓ Test before merging
- ✓ Tag every production release
- ✓ Automate tests and checks
- ✓ Document incidents and fixes
- ✓ Keep history clean and understandable



REMEMBER: Good Git workflows prevent problems. Good troubleshooting skills fix them fast.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta

VERIQTA SOCIAL



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

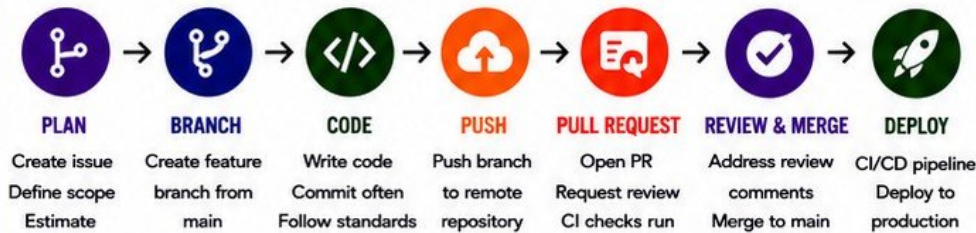


20. ENTERPRISE BEST PRACTICES AND FINAL REVIEW



Build secure. Ship reliably. Scale confidently.

1 END-TO-END GIT WORKFLOW



2 PRODUCTION CHECKLIST

- ✓ Use meaningful branch names
- ✓ Write clear commit messages
- ✓ Keep PRs small and focused
- ✓ Require code reviews
- ✓ Ensure CI/CD pipeline passes
- ✓ Run tests and security scans
- ✓ Update documentation
- ✓ Tag releases with versions
- ✓ Monitor deployments
- ✓ Have rollback plan ready
- ✓ Protect main and release branches
- ✓ Backup important repositories

3 COMMON MISTAKES TO AVOID

- ✗ Committing large or unnecessary files
- ✗ Pushing directly to main branch
- ✗ Writing unclear commit messages
- ✗ Skipping code reviews
- ✗ Ignoring CI/CD failures
- ✗ Not resolving conflicts properly
- ✗ Force pushing without understanding impact
- ✗ Committing secrets or credentials
- ✗ Not tagging releases
- ✗ Not synchronizing with main before PR
- ✗ Leaving stale branches
- ✗ No rollback plan for releases

4 ENTERPRISE STANDARDS

- ✓ Branch Protection: Protect main and release branches
- ✓ Code Review: Mandatory reviews for all changes
- ✓ Commit Messages: Follow Conventional Commits
- ✓ CI/CD: All changes must pass pipeline
- ✓ Security: No secrets in code or history
- ✓ Access Control: Least privilege access
- ✓ Documentation: Keep docs up to date
- ✓ Versioning: Use Semantic Versioning
- ✓ Monitoring: Track deployments and errors
- ✓ Compliance: Follow organizational policies

5 GIT COMMAND CHEAT SHEET

TASK	COMMAND
Clone repository	git clone <url>
Check status	git status
Add files	git add <file>
Commit changes	git commit -m "message"
Push changes	git push origin <branch>
Pull latest changes	git pull origin <branch>
Create branch	git branch <name>
Switch branch	git switch <name>
Merge branch	git merge <branch>
Stash changes	git stash
List stashes	git stash list
Apply stash	git stash apply
Revert commit	git revert <commit>
Reset (soft)	git reset --soft <commit>
Reset (hard)	git reset --hard <commit>
Show log	git log --oneline --graph --all

6 LEARNING ROADMAP

- 1 Git Fundamentals
Installation, configuration, basic commands
- 2 Working with Repositories
Cloning, committing, branching, merging
- 3 Advanced Git
Rebase, stash, cherry-pick, tagging
- 4 Collaboration
Pull requests, reviews, conflicts
- 5 GitHub Advanced
Actions, Projects, Security, Codespaces
- 6 CI/CD Integration
Automate build, test, and deployment
- 7 Enterprise Git
Policies, standards, large scale workflows
- 8 Mastery
Real-world projects, troubleshooting, optimization

7 SENIOR ENGINEER RECOMMENDATIONS

- ✓ Automate everything you can.
Reduce manual steps and human error.
- ✓ Make small, atomic commits.
Easier to review, test, and rollback.
- ✓ Communicate early and often.
Good collaboration prevents bad merges.
- ✓ Understand the impact before you push.
Think before you force push or rewrite history.
- ✓ Keep your local repository clean.
Sync often, prune branches, clear stashes.
- ✓ Document everything that matters.
Decisions, processes, and exceptions.
- ✓ Monitor and learn from every release.
Continuously improve your workflow.
- ✓ Security and quality are non-negotiable.
Protect your code, users, and organization.



FINAL THOUGHT

Great Git practices
lead to great software.
Master your workflow.
Collaborate effectively.
Ship with confidence.

**KEEP LEARNING.
KEEP BUILDING.
KEEP SHIPPING.**



**SUCCESS
FORMULA**



Clean Code

+



Collaboration

+



Security

+



Automation

=



Reliable Delivery



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

VERIQTA SOCIAL